cs-mail Rust Reference Architecture

C-SQD

Draft 0.11 — 8 October 2026

Abstract

This architecture follows one relationship request from charge preparation to recipient decision, then follows a forfeiture into the annual member program and one automatic bank payment of the member's allocation. A pure transition computes the required state and accounting obligations. A transaction commits them with an outbox; the configured processor confirms payment effects afterward. This division gives the revised protocol a small deterministic center while keeping delivery, payment uncertainty, and member eligibility in explicitly bounded services.

Design principles. Before changing the implementation or this design, read the Rust-domain design principles (`docs/rust-domain-principles.md`). They are binding for cs-mail and identity-model; new work must not regress them.

Status and reading order. The *Protocol Specification*, normative financial-program profile, and C-SQD domain model control the rules. This document describes the Rust libraries and durable boundaries. The local server and CLIs are runnable; the SwiftUI member application and live payment adapter remain product work. Request classes and atomic lane acceptance are implemented, including native lane sends. The implementation notes describe the coordinated format cutover.

1 Planned Apple Client Boundary

The 8 October 2026 client decision replaces the planned Tauri shell with SwiftUI. Ship macOS first, establishing iOS targets and shared Apple presentation code from the first slice. The application calls a narrow Rust client facade through Swift bindings. Introduce that facade and device/simulator library packaging during Milestone 4, alongside the first consumer; these are planned additions.

Server Rust remains authoritative for account, permission, settlement, billing and distribution decisions. Client Rust owns portable signing, encryption/decryption, protocol handling, synchronization and command-recovery workflows. Swift owns views, navigation, presentation state and Apple integration. Platform adapters implement the client's key-custody, persistence, transport and lifecycle ports; no business rule is duplicated in the bindings or interface.

The facade exposes bounded, checked values, distinct identifiers and typed error meanings, with explicit object lifetimes, asynchronous completion and cancellation. A cancelled UI task does not cancel an accepted server command; preserve its operation identity and reconcile uncertain outcomes. Calls must not block the UI, and Swift presentation updates return to the main actor.

Keychain backs key custody. SQLite holds ciphertext, key references, cursors, minimized relationship metadata and nonsecret preferences, never secret keys or a plaintext message cache. Authentication, retained-copy entitlement and scoped consent remain separate authorities. Suspension, unavailable keys and offline use are explicit states; notifications trigger synchronization rather than establish canonical outcomes. The server remains authoritative after quit/reopen.

Apple targets share suitable SwiftUI components and presentation models, adapting navigation and platform behavior. The guest request page keeps its own web components. Later Android and Windows clients reuse the Rust client through separate bindings, interfaces and adapters; their implementation choices require validation. See the Apple client architecture and the product build plan for sequencing and validation scope.

2 One Request, Three Messages, One Decision

Alice selects Bob's research class; signed terms fix the class, $C + S$, and policy versions. Creation records one payment intent. After finalized funding, submission commits encrypted content, delivery intent, a deadline, and one solicitation. The directed pair has one nonterminal request across devices and classes.

Bob permits two follow-ups. Each has its own ciphertext, authenticated declarations, and delivery state; allowance is consumed, but charge, history, and deadline do not change.

Standing or lane acceptance atomically commits its permission, one $C + S$ refund obligation, request closure, and expiry cancellation. Alice may use the permission while refund execution awaits processor confirmation.

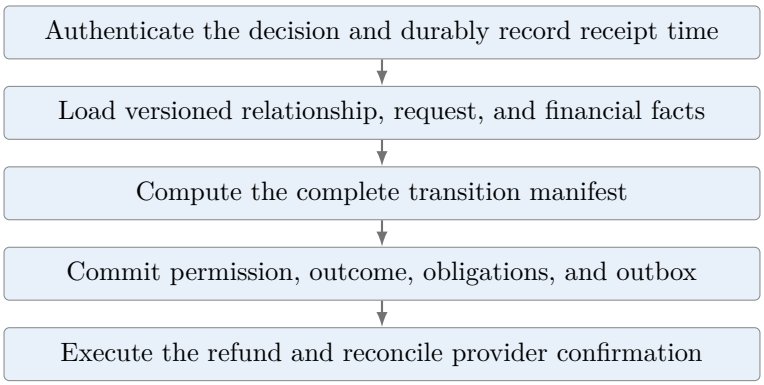


Figure 1: The commit establishes the obligation; external confirmation discharges it.

3 The Pure Transition and Its Durable Boundary

3.1 Authority enters as evidence

Wire decoding first checks versions, sizes, and canonical structure. Signature verification then binds the exact command digest to an authorized actor, key, deployment, provider, and receipt. Finally, domain conversion checks scope and command-specific constraints. An authenticated wrapper should have private constructors; a public boolean claiming authentication cannot carry this meaning.

The recipient's device time is not authoritative. Ingress durably records authenticated receipt before returning evidence to the client. That record lets a timely acceptance beat an expiry worker even when decision processing is delayed.

3.2 A snapshot contains the facts that can change

For acceptance, the snapshot contains the relationship, its current request if any, request-history context, relevant payment/refund facts, and ledger references. It carries versions and enough com-

pleteness evidence to distinguish no current request from a missing projection. Historical pending refunds may still exist after a request closes; they remain separate obligations rather than additional open relationship requests. Bounded acceptance also requires the authorized lane grant and protected capability state.

The pure function has a compact shape:

```
pub fn transition(
    snapshot: &SettlementSnapshot,
    authorized: &KernelCommand<ProtocolCommand>,
    context: &TransitionContext,
) -> Result<TransitionManifest, ProtocolError>;
```

It performs no I/O and reads no implicit clock or live policy service. Time, versions, immutable terms, and authenticated evidence are inputs. Its result describes the complete consequence, not just a new relationship-state enum.

3.3 The manifest describes the complete consequence

`TransitionManifest` contains expected snapshot and ledger revisions; the next protocol state, request history, payment records, and messages; a ledger batch; schedule changes; protocol events; outbox intents; and any terms result. Forfeitures and holds are explicit outputs as well. This lets one transition describe both permission and the money that follows from it.

An acceptance includes its required request outcome and refund obligation. The extension must coordinate the lane grant in that same commit for bounded acceptance, without changing the relationship to standing acceptance. A rejection records pending forfeiture without crediting the recipient. Follow-up admission cannot introduce another charge. The manifest's fields are public Rust data; correctness comes from the transition, validated commit path, and conformance tests, rather than a claim that private constructors alone prevent every inconsistent value.

Storage checks revisions and commits the manifest atomically. The PostgreSQL implementation uses transaction locks and version checks around the affected records. Application controllers do not recompute financial consequences after the kernel returns. Transactional outbox records make post-commit delivery and payment work recoverable.

4 Objects That Keep Different Lifecycles Apart

The relationship persists across requests. A request starts in `PreparingSubmission`. Its `SubmissionPreparation` records the deadline for completing funding and preparing the initial message. `CancelRequestSubmission` can cancel it during this phase.

Once those requirements are satisfied, `SubmitRequestToRecipient` commits the initial delivery. The lifecycle becomes `AwaitingRecipientDecision`; its `RequestSubmission` records submission time and decision deadline. Acceptance, rejection, and expiry close the submitted request. In the extension, an accepted request identifies standing permission or a lane; the latter concerns the relationship and is usually not scoped to one conversation. These names identify what is being prepared and whose decision is pending. "Admission" remains the broader per-message permission check.

A payment operation has a separate lifecycle. Pending external evidence is not finalized funding, and a refund can remain unpaid after its request is terminal. An annual allocation likewise remains owed after service coverage ends. `AnnualDistributionId` identifies the network allocation year; `BillingAccountId` identifies the person's bank-linked account; `ServiceContractId` identifies a

fixed service purchase and is distinct from the `PaymentOperationId` of a collection attempt. Neither is a public communication identity or a private anti-abuse principal.

Money uses integer minor units and an explicit settlement unit. Checked arithmetic rejects overflow and mixed-unit operations. Corporate rates use exact basis-point arithmetic. Submission deadlines, decision deadlines, message validity, annual allocation cutoffs, service intervals, and payment dates answer different questions and remain explicit inputs.

Quoted financial terms stay fixed for a request. Pricing is set by the operator's single versioned `RequestPricingPolicy`: processing charge and collateral bounds. `RecipientRequestClasses` belongs to one protocol identity and checks at most eight user-defined classes and unique IDs, including on deserialization. `RequestPricingService` authenticates a publication against a key of that address alone, under a `RecipientSigningScope` that names no relationship, and rejects collateral outside the current bounds. `QuotePricing::resolve`, the single pricing rule used by every storage adapter, yields a checked `RequestPrice` or a `PricingRefusal` (no policy, unknown class, or a class now outside the bounds). The kernel receives it as `TransitionContext::pricing`, not as part of the policy snapshot, and surfaces a refusal only when a charge is required. Signed terms retain the policy version, C, S and a `SelectedRequestClass` snapshot. There is no fallback to caller-supplied prices or an invented default class. Class edits do not rewrite existing quotes or partition principal-recipient history. Current follow-up policy is checked separately for each message.

5 Payment Work Is a Recoverable Conversation

5.1 Create before dispatch

Creation claims the unique nonterminal request for the directed pair and records one authorized payment intent before calling a provider. A database uniqueness constraint or equivalent aggregate guard prevents two devices with different keys from dispatching two charges for the same pending relationship request. The worker's external idempotency key derives from that committed operation, not from a fresh network retry.

An external timeout means uncertainty. The worker queries the original operation or repeats it under the provider's idempotency contract; it cannot assume failure and invent a replacement. Submission waits for finalized funding evidence and durable content. The platform does not rely on an authorization remaining open throughout the decision window.

5.2 New funding attempts require separate authority

`PaymentOutcome::Pending` records uncertainty; `PaymentOutcome::Settled` represents authenticated satisfaction of the provider's funding-finality policy. A real adapter must map rail-specific evidence to that contract. Authorization or receipt of a provider request alone is insufficient. The current simulator can delay resolution, fail an operation, lose a response, or report a later reversal.

`FundingSource` in `cs-mail-finance` owns verification, restriction, and attempt accounting. PostgreSQL shares the source record across an account's request and utility charges. Reserving a new operation consumes available attempt capacity; reconciling the same uncertain operation does not. Failure or reversal restricts the source. `maximum_unresolved` bounds simultaneous unresolved attempts, not lifetime attempts. Authenticated bank evidence permits re-verification of the same person and route; bank replacement remains deferred. An explicit account command can request a collection retry while retaining the contract's terms. A worker cannot manufacture authority with another operation ID.

A reservation is not a dispatch grant. The PostgreSQL `authorize_request_dispatch` and `authorize_utility_dispatch` transactions check current source restrictions before durably granting a new dispatch. Previously authorized work remains reconcilable under its operation identity. Restricted queued work waits for re-verification. A Rust value created earlier cannot prove that mutable external restrictions have remained unchanged.

5.3 Cancellation can precede confirmation

If submission timeout or sender cancellation commits first, the request stays cancelled. The payment adapter voids an uncaptured authorization or refunds a capture. If a capture confirmation arrives afterward, the reconciler records the financial fact and schedules the full refund without reopening the request. This recovery completes the existing obligation after the request has closed.

`PaymentProcessor` exposes submission, lookup, and explicit `cancel_capture` with a validated `CaptureCancellation`. Cancellation reaches a pending capture rather than being consumed by lookup. `SimulatedProcessor` exercises these contracts; one configured processor authority serves the deployment. This trait is not a multi-processor router.

Provider notifications are authenticated evidence. Deduplicate them by provider event and economic operation, check amount, unit, and destination scope, and reconcile out-of-order facts without regressing confirmed state. Both polling and webhooks may discover the same event; neither may duplicate a refund.

5.4 Completion is separate from obligation

Refund and payout workers read durable obligations and execute stable operations. Failure leaves an obligation pending or under review. Success records reconciled confirmation and the corresponding ledger effect. A chargeback or adjudicated correction appends a separately authorized batch. It does not overwrite a recipient decision or confiscate a finalized member allocation.

The ledger therefore projects utility-service receipts, request charges, refunds due and completed, pending forfeitures, corporate revenue, restricted member funds, and payables. It does not expose a user wallet. A refund or payable cannot be applied as available funds for another request.

6 From a Rejection to an Annual Allocation

Bob's rejection records processing revenue C and a pending forfeiture lot for S. The lot retains the request's quoted rate version. Maturity evaluation is a separate decision over verified financial facts and policy; a calendar boundary does not make unresolved funds distributable.

`cs-mail-finance` owns the financial program. An `AnnualDistributionSchedule` fixes a UTC calendar year and eligibility policy. At finalization, `FinancialProgram` fixes qualifying membership and eligible funding, assesses each new lot once, and records corporate and member amounts, equal individual payables, and restricted remainder. It consumes no renewal or billing status when deciding eligibility. The deployment profile defines the arithmetic; this domain supplies its single implementation.

The current PostgreSQL finalizer loads the program records and ledger under a transaction lock and commits the allocation as one outcome. Replay reuses the finalized result. It does not implement chunked preparation for large membership sets. A future bounded-batch design must preserve the same atomic visibility and once-only assessment guarantees.

A `MemberPayable` belongs to one member and pool year. Failed or delayed payments stay assigned, outside the next year's equal split. Its private amount and published `DistributionTerms`

determine reporting values $R_i = \min(D_i, U)$ and $X_i = \max(0, D_i - U)$. Those terms snapshot the annual utility amount and due date independently of renewal. The payable owns at most one `PaymentExecution`, never separate rebate and cash executions. Authorized corrections append records with a stated cause; they do not rewrite the annual snapshot or create another ordinary annual allocation.

7 Billing Owns Fixed Service Purchases and Coverage

`cs-mail-billing` owns the bank-linked `BillingAccount`. It publishes `ServiceOffer` values and records each purchase as a `ServiceContract`. A validated `ServicePeriod` spans one calendar/anniversary year with an explicit leap-day rule. An offer pins its policy version, price, settlement unit, collection date, and interval. Its constructor requires collection before the interval starts; the product requires collection well in advance, with the exact lead time supplied by policy. No universal date or number of days is invented.

Purchase snapshots those terms into a contract. Collection retries preserve price and interval; they never start a fresh year from the retry date. Coverage requires finalized funding, the current time within that fixed interval, and funding already completed at that time. Late funding grants no past service. Distribution does not alter the contract or revoke coverage.

`BankVerification` binds member, account, bank token, scope, unit, and verification version under the configured bank authority. Verification produces an opaque `VerifiedBankAccount`, not a deserializable claim of verification. Trusted registration enforces member and alias ownership and explicit account actor grants. Whether a person may hold a second account is decided by the identity service under cs-mail's evidence policy, with overlaps held for operator review; cs-mail keeps no person evidence. Accounts approved to share a bank account each keep their own funding source. Bank replacement and broader account-management UX remain deferred; this boundary is not a live bank connector.

`SignedBillingCommand` binds account, scope, revision, idempotency, and command to an active operational key under a distinct signing context. Explicit account grants authorize inspection, purchase of a published offer, collection retry, and closure. These commands cannot choose a processor, set policy prices, or supply payout instructions. There is no independent user-managed billing key. Closure stops coverage but preserves the bank association needed for obligations.

8 The Application Prepares One Automatic Distribution

Finance owns eligibility, pool-year allocation, published distribution terms, and the resulting obligation. Billing supplies the verified bank association. The pure coordinator, `prepare_distribution`, checks that association, scope, unit, payable, and due time, then prepares one bank payout of the full allocation. It does not examine renewal or available refund capacity. Pending next-year collection and account closure do not gate this payment.

`PaymentExecution` retains sequential attempts and authenticated events. Its states distinguish pending, settled, failed, voided, and reversed, with the required evidence carried by each state. A lost response reuses the operation identity. Only a definite failed or voided attempt permits a replacement attempt; conflicting evidence is rejected. Reversal retains settlement and reversal evidence and the unpaid obligation; it is not silently reissued.

PostgreSQL uses the same coordinator, commits payable changes and work together, and loads the associated bank route rather than user-supplied distribution instructions. Annual finalization atomically queues automatic preparation; workers wait for the published due time. Preparing one

member's payment uses selected owner records rather than loading the population. Complete-program operations, such as annual finalization, reject such incomplete views.

Service collection and distribution remain separate transactions and worker operations. One distribution has one completion status; rebate and excess have no independent retries. Statements derive the two reporting amounts from the fixed allocation and utility amount, including after closure or nonrenewal.

9 Admission, Scheduling, and Concurrency

The host checks funded local service coverage before new communication. This applies to the local recipient and any local sender, including accepted and lane paths; a remote sender's provider owns its coverage check. Coverage expiry leaves recipient decisions, financial recovery, and finalized allocations available.

All delivery paths observe blocking under the directed-pair transaction boundary. Standing acceptance and valid lanes authorize delivery without a request charge. An authenticated follow-up additionally identifies a request awaiting a decision and consumes the recipient's current allowance. Only eligible new-request creation may start a new payment operation, and it requires explicit sender authorization.

Follow-up policy updates, request closure, and delivery admission serialize. Content and a retryable delivery effect must be durable before admission commits. Message validity is checked independently of the request's common deadline. Lanes retain their signed scope and hard validity limits. Bounded acceptance must commit lane authority with request settlement; later lane changes do not reopen the outcome or create member activity. Independent issuance during a pending request and bounded-acceptance timing edge cases remain open in the domain model.

Scheduled work includes submission timeout, request expiry, payment reconciliation, annual allocation, member payment preparation, and retention. Maturity evidence must be recorded through the financial program before allocation. Each job carries a target, earliest execution time, version/evidence preconditions, and stable operation identity. Leased workers submit commands against current state. They do not assume that a row remains unchanged because a job was queued.

Principal-scoped new-request history needs its own concurrency guard shared by aliases. It advances only at initial submission. A follow-up, cancelled preparation, key rotation, or class switching cannot bypass that distinction. Existing late standing acceptance changes future permission while old expiry processing preserves that permission and materializes only the old request's financial obligation.

10 Responsibility Boundaries in the Workspace

The dependency direction follows ownership. Primitives provide money, time, and scoped identifiers. The ledger validates balanced batches. Finance depends on those foundations for payment evidence, request finances, forfeiture maturity, member eligibility, annual allocation, and payables. Billing builds on financial operations to own fixed service purchases and coverage. It does not decide who qualifies for a share.

Protocol owns request and relationship transitions and request pricing. Capabilities own lane grants and allowance. Security verifies communication and pricing authority; billing commands have their own signing contract. The application layer coordinates complete use cases, including the billing/finance automatic distribution, without acquiring either domain's policy decisions. For

bounded acceptance, capabilities validate the grant, protocol determines settlement, and application coordination commits both atomically. A grant also resolves a pending request when issued separately: preparation is cancelled with void/full refund, and a submitted request still pending is accepted even after its deadline. Terminal settlements stay fixed; future access follows the lane.

PostgreSQL owns durable transactions, ordering, uniqueness, journals, and outbox work. `PostgresDeployment` holds deployment-wide state—configuration, accounts, billing, the financial program, request pricing and deployment-owned work—and opens a `PostgresEngine` for each relationship aggregate. Workers lease that work, call domain operations through the application or storage boundary, and reconcile external evidence. Payment adapters execute operations and translate provider evidence; they cannot set eligibility, invent new funding authority, or change request outcomes.

Content and wire crates retain encryption and canonical declaration binding. The service exposes authenticated operations, and the client signs commands and consumes outcomes. Product interfaces build on these APIs rather than reconstruct settlement from presentation state. API, wire, event, protocol, database, and financial-policy versions remain distinct. Separate processes or credentials may be needed for privacy and operational isolation; a crate boundary alone supplies neither.

11 Privacy Extends to the Member Program

No routine component needs the entire story. The identity service knows control and recovery evidence. The relationship service sees scoped permission and request state. Content storage sees ciphertext. The ledger sees scoped financial causes and amounts. Membership verification sees the evidence needed to establish one qualifying member, while the allocation engine can consume a minimal eligibility assertion. Telemetry receives short-lived, rotated references.

Use separate access roles, keys, and credentials, including for financial and membership records that must outlive ordinary delivery data. Public identities and private principals must not become universal ledger or telemetry join keys. Native content is encrypted at endpoints; a payment or membership check does not authorize provider decryption. Authenticated purpose and origin remain accountable declarations, not proof of plaintext truth.

Eligibility activity should record only the minimal facts required by policy. Background synchronization and inbound lane traffic are not substitutes for intentional use. The payment processor receives required transaction references and amounts, not message content or the user's unrelated relationships.

Retention classes define purpose, key scope, operational lifetime, dispute or required retention, and deletion method. A settled request can lose unnecessary message linkage while an unpaid refund or member payable retains enough evidence for payment and audit. Continuing permission does not justify retaining its formation history forever. Cryptographic erasure must cover wrapped keys, replicas, backups, logs, and derived indexes; separately retained records must not be described as erased or two-party-only archives.

Logs and errors use safe event types and redacted identifiers. They must contain no plaintext, secrets, complete financial snapshots, or private identity evidence. Pairwise scoping reduces direct linkage but does not eliminate timing or amount correlation, so retention and access discipline remain necessary.

12 Version Boundaries and Evidence of Correctness

The clean domain update directly replaces the old billing cycles, split payments, and distribution instructions. It has no compatibility aliases or fallback readers. Wire format is 6, stored protocol format is 8, and stored financial-program format is 6. The revised development schema requires fresh initialization; populated older records are not converted by the migrations.

Private domain fields and checked constructors prevent invalid combinations in normal use. Stored rehydration additionally validates contracts, executions, payables, funding sources, request finances, and program records against their scope, ownership, arithmetic, evidence, and ledger. Durable uniqueness, dispatch checks, idempotency, and worker fencing address facts that local Rust types alone cannot guarantee.

The first story test uses one charged request and three permitted messages. Acceptance creates one refund obligation, and provider confirmation completes it once. Variants reject into pending forfeiture, expire into a collateral refund, disable follow-ups, and race policy changes with admission. Other tests exercise two-device creation, alias history, cancellation followed by late capture, duplicate/out-of-order notifications, and unknown commit recovery. Existing scenarios now exercise fixed class terms, publication limits, lane/refund rollback and retry, preparation and late grants, and native lane allowance. Shared history and terminal settlement remain domain invariants; passing scenarios are limited evidence, not exhaustive proof.

Financial-program tests reproduce a year from fixed input manifests, exclude refundable and immature funds, assess each eligible lot once, preserve quoted rate cohorts, and allocate equal integer shares. Include zero members, small amounts, unallocated remainder, failed payments, and annual finalization retries after a crash. A later year must not assess old member funds or redistribute already assigned obligations. Billing tests cover advance collection, fixed-period retries, leap-day rules, pending funding, and coverage boundaries. Domain and PostgreSQL acceptance stories cover one verified person/account, explicit actor grants, current restrictions on queued work, automatic payment after closure with next-year funding pending, and lost-response recovery for one lump sum. With $U = \$120$, \$80/\$120/\$170 allocation examples verify reporting without separate payment components. Corrupt evidence, malformed stored owners, and incomplete program views are rejected.

Property and concurrency tests check conservation, finality, uniqueness, scope, and recovery at durable boundaries. Privacy tests exercise cross-domain credential denial, captured-log redaction, and deletion through backup restoration. Fuzz canonical decoding, signatures, and content bindings.

Independent-provider federation follows only after a financial agreement assigns pool governance, membership uniqueness, refund and payout obligations, clearing, and losses. The local manifest remains useful as a reference outcome, but the old direct-recipient clearing model is not carried forward automatically.